Inventory & Order Management SaaS using ASP.NET Core + React, I would design it as a proper multi-tenant SaaS from the beginning rather than a simple CRUD inventory application.

Inventory & Order Management SaaS — Functional Requirements

1. Product Overview

The system should allow businesses to manage:

- Products
- Categories
- Warehouses
- Inventory
- Customers
- Suppliers
- Sales orders
- Purchase orders
- Stock transfers
- Payments
- Invoices
- Users and permissions
- Reports
- Notifications
- Business settings

The core workflow should be:

Products
↓
Purchasing → Warehouse → Inventory
↓
Sales Orders
↓
Invoice / Payment
↓
Customer

---

2. Multi-Tenant SaaS

This is **very important** if you're selling the product to multiple businesses.

Each business should have its own isolated data.

Organization

A user can create or belong to an organization.

```
Platform
├── Company A
│   ├── Users
│   ├── Products
│   ├── Customers
│   ├── Warehouses
│   └── Orders
│
├── Company B
│   ├── Users
│   ├── Products
│   ├── Customers
│   └── Orders
```

Requirements

- Create organization
- Organization profile
- Company logo
- Business address
- Tax information
- Currency
- Time zone
- Fiscal year
- Number/date formats
- Organization status
- Subscription plan
- Subscription limits

Every important database record should be associated with:

TenantId

---

3. Authentication & Authorization

Authentication

- Register

- Login
- Logout
- Forgot password
- Reset password
- Email verification
- Change password
- Refresh token
- Session management

Authorization

Use RBAC.

Example roles:

Super Admin
↓
Company Admin
↓
Manager
↓
Salesperson
↓
Warehouse Staff
↓
Accountant

Permissions could include:

Products.View
Products.Create
Products.Update
Products.Delete

Orders.View
Orders.Create
Orders.Update
Orders.Cancel

Inventory.View
Inventory.Adjust

Reports.View
Users.Manage

---

4. Product Management

This is one of the main modules.

Product

Each product should support:

- Product name
- SKU
- Barcode
- Description
- Category
- Brand
- Unit of measure
- Cost price
- Selling price
- Tax
- Minimum stock
- Maximum stock
- Reorder level
- Product image
- Status
- Weight
- Dimensions

Product variants

For example:

Nike T-Shirt

Color:
- Black
- White
- Blue

Size:
- S
- M
- L
- XL

Each variant can have:

- SKU
- Barcode
- Price
- Cost
- Stock

- Image

---

5. Categories & Brands

Categories

Support hierarchical categories:

Electronics
├── Mobile Phones
├── Laptops
└── Accessories

Functions:

- Create
- Edit
- Delete
- Activate/deactivate
- Parent category

Brands

- Create brand
- Edit brand
- Delete brand
- Product association

---

6. Warehouse Management

Businesses may have multiple warehouses.

Example:

Company

Warehouse A — Islamabad
Warehouse B — Rawalpindi
Warehouse C — Lahore

Each warehouse should have:

- Name
- Code
- Address
- Contact
- Manager
- Status

---

7. Inventory Management

This is the heart of the system.

The system should track:

Opening Stock
+ Purchased Stock
+ Transfer In
+ Returns
- Sales
- Transfer Out
- Damaged Stock
= Available Stock

Inventory dashboard

Show:

- Total products
- Total stock quantity
- Low-stock products
- Out-of-stock products
- Inventory value
- Stock movement
- Fast-moving products
- Slow-moving products

---

8. Stock Movements

Every inventory change should create a movement record.

Example:

Product: iPhone 17

Warehouse: Islamabad

+20 Purchase
-3 Sales
-2 Transfer
+1 Customer Return
-1 Damaged

Current Stock = 15

Movement types:

- Purchase
- Sale
- Sales return
- Purchase return
- Transfer
- Adjustment
- Damage
- Opening stock

This gives you a complete **inventory audit trail**.

---

9. Stock Adjustment

Authorized users should be able to adjust inventory.

Example:

System says:

Expected: 100
Physical: 97
Difference: -3

User creates adjustment:

Adjustment:
Product: ABC
Quantity: -3
Reason: Damaged

The system records:

- User
- Date/time

- Warehouse
- Previous quantity
- New quantity
- Reason

---

10. Stock Transfer

Transfer products between warehouses.

Example:

Islamabad Warehouse
↓
Transfer
↓
Lahore Warehouse

Status:

Draft
↓
Requested
↓
Approved
↓
In Transit
↓
Received

---

11. Customer Management

Customer fields:

- Customer code
- Name
- Company
- Email
- Phone
- Address
- City
- Tax number
- Credit limit
- Payment terms

- Customer status

Customer profile should show:

Customer
├── Orders
├── Invoices
├── Payments
├── Returns
├── Outstanding Balance
└── Activity History

---

12. Supplier Management

Similar to customers.

Supplier:

- Name
- Code
- Contact
- Email
- Phone
- Address
- Tax information
- Payment terms
- Outstanding balance

Profile:

Supplier
├── Purchase Orders
├── Purchase Invoices
├── Payments
└── Purchase Returns

---

13. Sales Order Management

This should be a major module.

Create Sales Order

Customer
Warehouse

Order Date

Products
├── Product
├── Quantity
├── Price
├── Discount
├── Tax
└── Total

Subtotal
Discount
Tax
Shipping
Grand Total

Order statuses

Draft
↓
Confirmed
↓
Processing
↓
Packed
↓
Shipped
↓
Delivered

Also:

Cancelled
Returned
Partially Returned

---

14. Order Approval

For larger businesses, introduce approval workflows.

Example:

Salesperson creates order
↓
Manager approval
↓

Warehouse processing
↓
Shipment

Rules can be configured:

Order > Rs. 500,000
↓
Manager approval required

---

15. Purchase Orders

Purchase workflow:

Purchase Request
↓
Purchase Order
↓
Supplier
↓
Goods Received
↓
Inventory Updated
↓
Purchase Invoice
↓
Payment

Purchase order should contain:

- Supplier
- Warehouse
- Products
- Quantities
- Prices
- Tax
- Discount
- Expected delivery
- Notes

---

16. Goods Receiving

Warehouse user receives purchased products.

Example:

PO #10025

Ordered:
100 units

Received:
95 units

System should support:

- Full receiving
- Partial receiving
- Damaged quantity
- Rejected quantity

Inventory should only increase by the **actual received quantity**.

---

17. Sales Returns

Customer returns products.

Workflow:

Sales Order
↓
Return Request
↓
Inspection
↓
Approved
↓
Inventory Restored

Return reasons:

- Damaged
- Wrong product
- Wrong size
- Customer changed mind
- Defective

---

18. Purchase Returns

Business returns products to supplier.

Purchase
↓
Return
↓
Supplier

Inventory decreases accordingly.

---

19. Pricing Management

You can make this feature very powerful.

Support:

Base price

Cost = Rs. 700
Selling = Rs. 1,000

Customer-specific pricing

Retail Customer → 1,000
Wholesale → 900
Distributor → 850

Quantity pricing

1–9 → 1,000
10–49 → 900
50–99 → 850
100+ → 800

This could become a major selling point for distributors.

---

20. Discounts

Support:

- Percentage discount

- Fixed discount
- Product discount
- Order discount
- Customer-specific discount
- Promotional discount

Example:

Product = 10,000

10% discount
= 9,000

---

21. Tax Management

Support configurable taxes.

Example:

GST
Sales Tax
VAT
Custom Tax

Tax can be configured at:

- Product level
- Customer level
- Order level

---

22. Invoices

Generate invoices from sales orders.

Invoice:

Invoice #INV-10025

Customer
Products
Subtotal
Discount
Tax

Shipping
Grand Total
Paid
Remaining

Generate:

- PDF
- Print
- Email
- Download

---

23. Payments

Track payments.

Payment methods:

- Cash
- Bank
- Card
- Online payment
- Other

Payment status:

Unpaid
Partially Paid
Paid
Overdue

---

24. Accounts Receivable

Customer balance:

Total Sales 500,000
Paid 350,000
Outstanding 150,000

Show:

- Outstanding invoices
- Due dates

- Overdue invoices
- Payment history
- Customer credit limit

---

25. Dashboard

The main dashboard should give management a quick overview.

```
┌──────────────────────────────────────────────────┐
│ Sales Today          Rs. 450,000        │
│ Orders Today          127               │
│ Products             8,542              │
│ Low Stock             34                │
└──────────────────────────────────────────────────┘
```

Sales Trend


Top Products
1. Product A
2. Product B
3. Product C

Recent Orders
#1001
#1002
#1003

---

26. Reports

Sales reports

- Daily sales
- Monthly sales
- Sales by product
- Sales by category
- Sales by customer
- Sales by salesperson
- Sales by warehouse

Inventory reports

- Current stock
- Stock valuation
- Stock movement
- Low stock
- Dead stock
- Fast-moving products
- Slow-moving products

Purchase reports

- Purchases by supplier
- Purchase trends
- Purchase by product

Financial reports

- Customer receivables
- Supplier payables
- Profit/loss summary
- Tax summary

Export:

Excel
CSV
PDF

---

27. Notifications

Notifications should be generated for:

- Low stock
- Out of stock
- New order
- Order approval
- Order cancellation
- Payment received
- Invoice overdue
- Purchase order received

Channels:

In-app
Email
WhatsApp

---

28. Audit Logs

Very important for a business application.

Track:

User
Action
Entity
Old Value
New Value
IP
Date/Time

Example:

Hamza changed Product ABC price

Old: 10,000
New: 11,000

25-Aug-2026 10:42 AM

---

29. Search & Filtering

Every major screen needs:

- Search
- Pagination
- Sorting
- Date filtering
- Status filtering
- Warehouse filtering
- Customer filtering
- Export

For example:

Orders
_____

Search: [ORD-10025]

Status: [Confirmed ▼]
Date: [01-Aug] - [25-Aug]

30. SaaS Subscription

Since this is a SaaS product, add subscription management.

Example:

Starter

- 2 users
- 1 warehouse
- 1,000 products

Business

- 10 users
- 5 warehouses
- 20,000 products

Enterprise

- Unlimited users
- Unlimited warehouses
- API access
- Advanced reports
- Integrations

You can later integrate Stripe or another payment provider depending on your target markets.

---

31. API & Integrations

This can become a huge differentiator.

REST API

External systems can access:

/products
/customers
/orders

/inventory
/warehouses
/invoices
/payments

Integrations

Eventually:

- Shopify
- WooCommerce
- WhatsApp
- Amazon
- POS systems
- SAP Business One
- QuickBooks
- Xero

Given your existing Shopify/SAP experience, I'd make **Shopify + SAP Business One integration** a premium feature.

---

32. AI Features — Phase 2

Don't make AI the foundation of version 1.

Once the core system works, add:

AI inventory assistant

User asks:

> "Which products are likely to run out next week?"

AI analyzes inventory and sales history.

AI sales assistant

> "Show me customers who haven't purchased in 90 days."

AI reporting

> "What were our best-selling products this month?"

Demand forecasting

Historical Sales

↓
AI/ML Model
↓
Expected Demand
↓
Recommended Purchase Quantity

Recommended MVP

Don't build all 32 modules initially.

Your first commercially usable version should contain:

Phase 1 — MVP

Authentication
Multi-tenancy
Users & Roles

Products
Categories
Warehouses

Inventory
Stock Movements
Stock Adjustments
Stock Transfers

Customers
Suppliers

Sales Orders
Purchase Orders

Invoices
Payments

Dashboard
Basic Reports
Audit Logs

That's enough to start showing the product to real businesses.

Phase 2

Returns
Advanced pricing

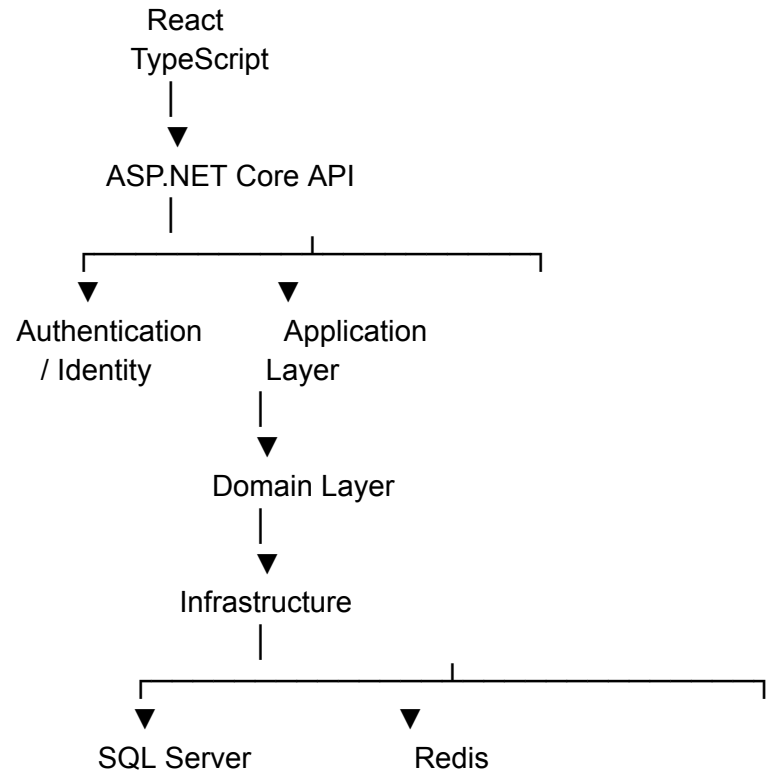
Approval workflows
Advanced reports
Notifications
PDF/Excel exports
Subscriptions
Public API

Phase 3

Shopify
WooCommerce
WhatsApp
SAP Business One
AI Assistant
Demand Forecasting
Automated Reordering

Suggested .NET + React architecture

For your skill set, I'd use:



And structure the backend roughly as:

```
src/
├── API
├── Application
```

```
├── Domain
├── Infrastructure
└── Shared
```

For the frontend:

```
src/
├── components/
├── pages/
├── features/
│   ├── products/
│   ├── inventory/
│   ├── orders/
│   ├── customers/
│   ├── suppliers/
│   └── reports/
├── services/
├── hooks/
├── store/
├── types/
└── utils/
```

One architectural decision I'd make early: use `TenantId` throughout the data model and enforce tenant isolation at the application/data-access level. That prevents the biggest class of SaaS security problems later.

If you're actually going to build this, the next useful step is to turn these requirements into a **complete FSD + database ERD + API endpoint specification + React page/module structure**, before writing the .NET code.